WORKSHEET 3

Student Name: Analava Bera **UID: 23BCS13611**
Branch: BE-CSE **Section/Group: Krg_3A**
Semester: 6th **Subject Name: System Design**
Subject Code: 23CSH-314

Aim: To design a social media application (Instagram/Facebook)

Steps:

1. **Functional Requirements:**
   A. Client should be able to register and login to the application.
   B. Client should be able to create post (text / image / videos)
   C. Client should be able to follow each other (or send friend requests)
   D.Client should be able to like or comment on the post
   E. Client should be able to view the feed of post from users they follow.

2. **Non-Functional Requirements:**
   A. Scalability: 500M DAU.
   B. Consistency & Availability:
   In this case -> Social Media
   1. We need our application to be HIGHLY available, then consistent
      Reason: If it is not operational / Functional when it was required, then there is no
      meaning of developing the application.
   C. Latency: (Uploading speed of publishing post): 500ms to upload post

3. **API Design:**

   A. User On-boarding API's
   1. **User Registeration:** POST API CALL: POST / api / users / register_user
   2. **User Login: POST API CALL:** POST / api / users / login
   3. **User Data Display: GET API CALL:** GET / api / users / {user_id} / profile
   4. **User Data Update: PUT API CALL:** PUT / api / users / {user_id} / profile

B. User Post's

1. POST / api / user_id / posts
2. GET / api / posts / {post_id}
3. PUT / api / posts / {post_id}
4. DELETE / api / posts / {post_id}
5. GET / api / posts / feed / limit = {limit} & offset = {offset} : PAGINATION
6. GET / api / users / {user_id} / posts: PAGINATION
   For a particular user if we want to fetch the post's

C. User Interactions

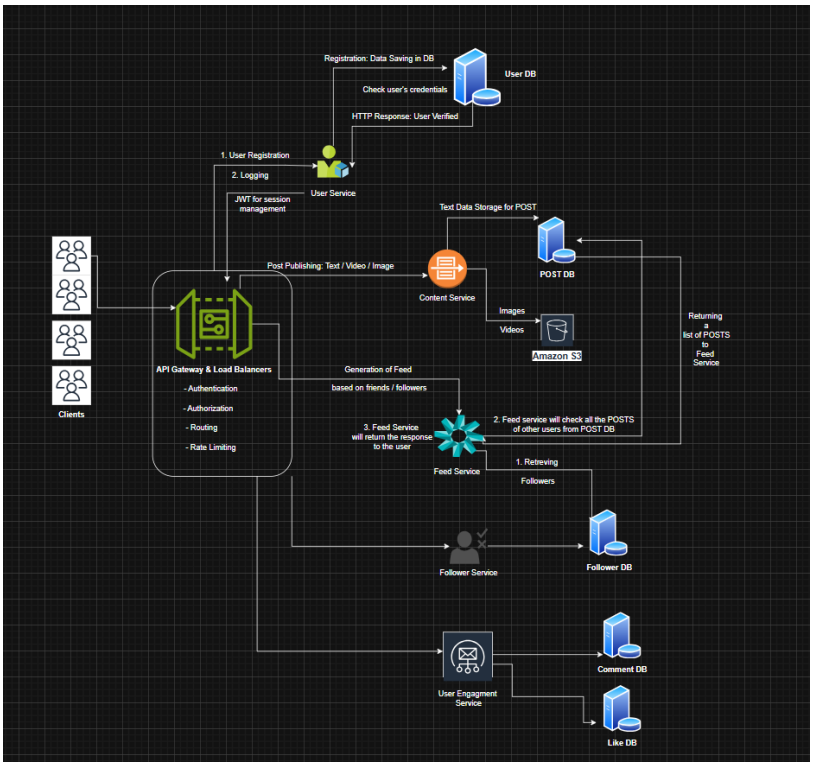
1. POST / api / posts / {post_id} / like
2. DELETE / api / posts / {post_id} / unlike
3. POST / api / posts / {post_id} / comments
4. GET / api / posts / {post_id} / comments
5. PUT / api / comments / {comment_id}
6. DELETE / api / comments /{post_id}/ {comment_id}
7. POST / api / users / {user_id} / follow
8. DELETE / api / users / {user_id} / unfollow

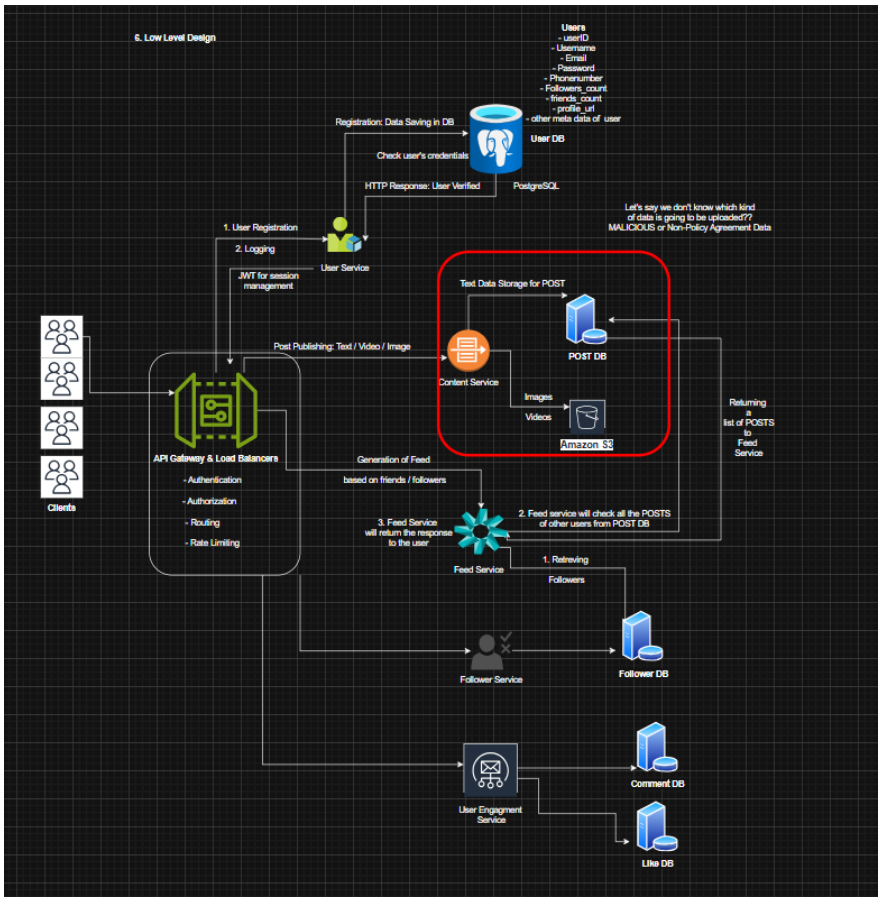
4. High-Level Design:

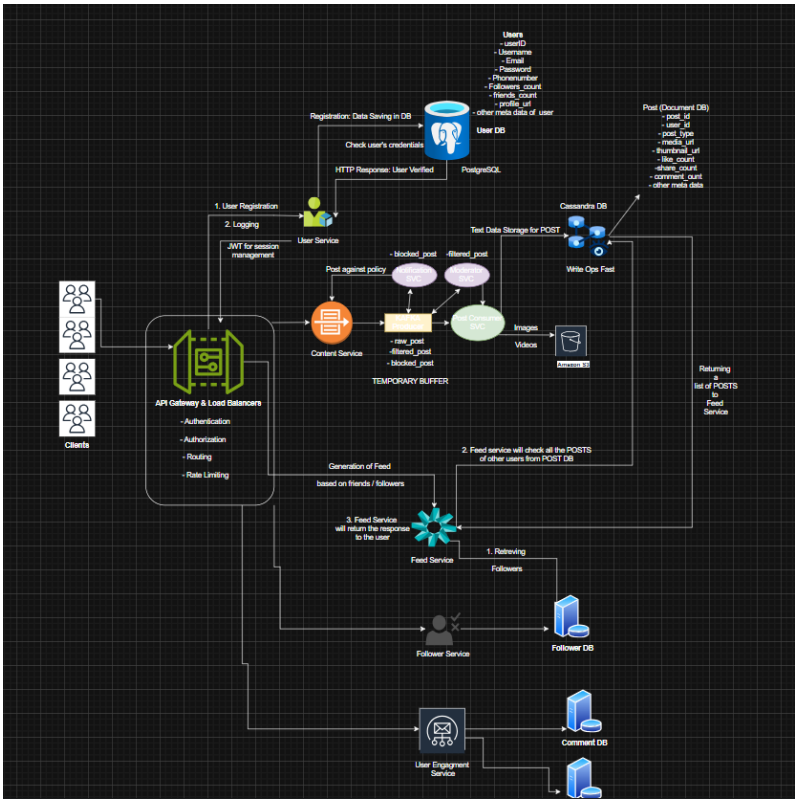
Now According to the functional requirement of the system, we can identify that :

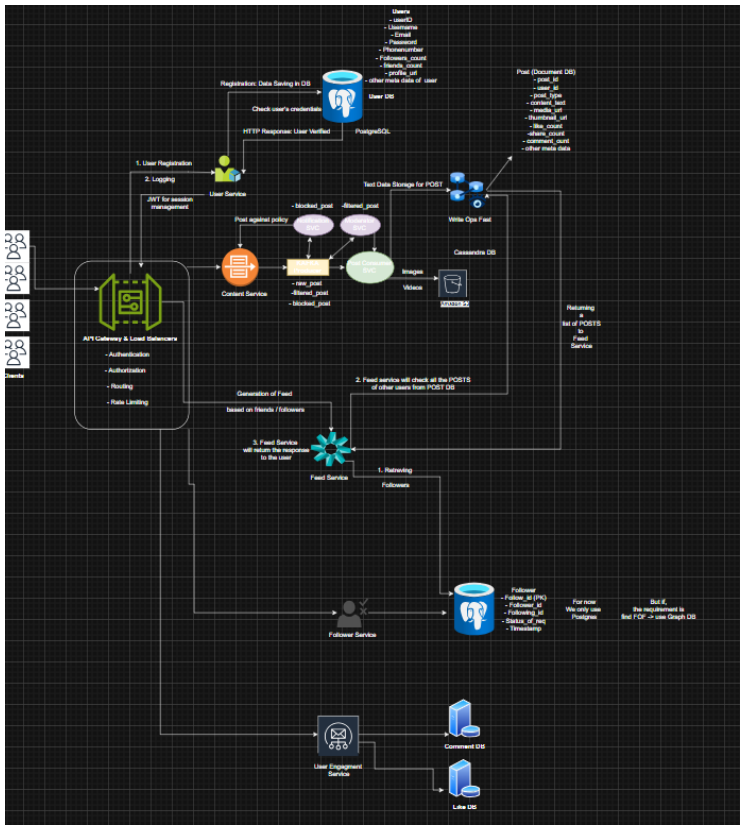
There will be a client who is requesting, then there will a server upon which computation will be going on, and lastly there will be an database in which storage will be done.

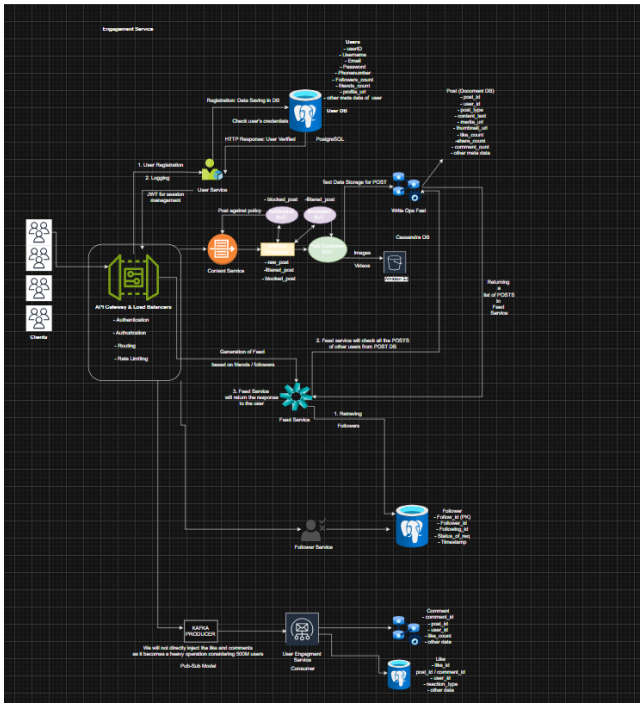
5. Low-Level Design:

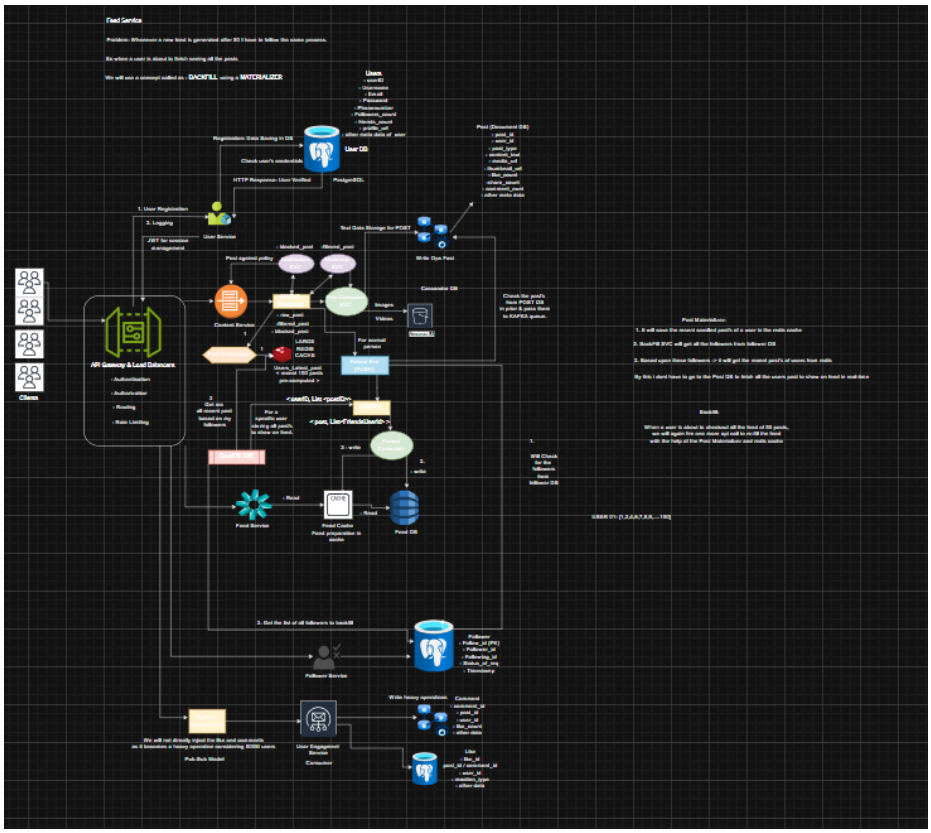


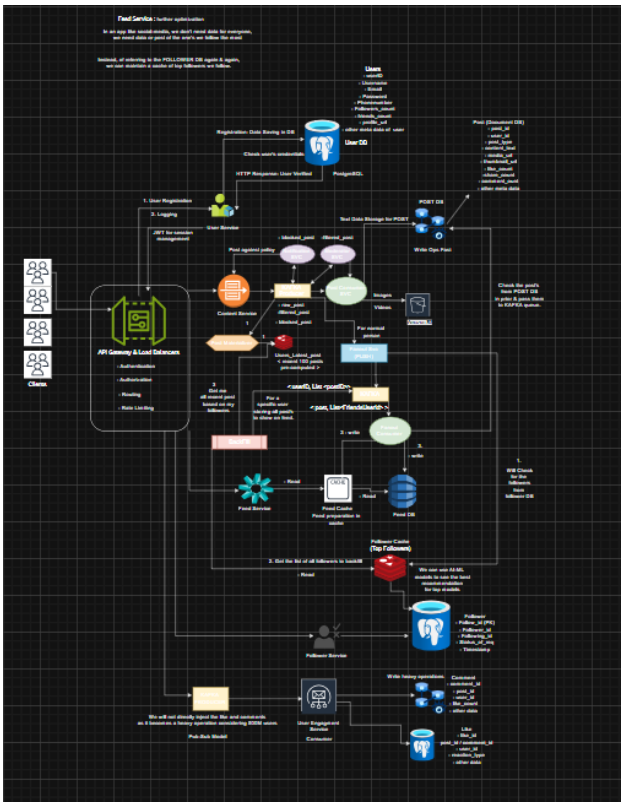

5. Low Level Design
Users
- userID
- Username
- Email
- Password
- Phonenumber
- Followers_count
- friends_count
- profile_url
- other meta data of user
User DB
PostgreSQL
Registration: Data Saving in DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Let's say we don't know which kind of data is going to be uploaded?? MALICIOUS or Non-Policy Agreement Data
Text Data Storage for POST
POST DB
Content Service
Post Publishing: Text / Video / Image
Images
Videos
Amazon S3
Returning a list of POSTS to Feed Service
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Generation of Feed
based on friends / followers
2. Feed service will check all the POSTS of other users from POST DB
3. Feed Service will return the response to the user
Feed Service
1. Retreving
Followers
Follower Service
Follower DB
User Engagment Service
Comment DB
Like DB


Users
- userID
- Username
- Email
- Password
- Phonenumber
- Followers_count
- friends_count
- profile_url
- other meta data of user
User DB
PostgreSQL
Registration: Data Saving in DB
Check user's credentials
HTTP Response: User Verified
Post (Document DB)
- post_id
- user_id
- post_type
- media_url
- thumbnail_url
- like_count
-share_count
- comment_count
- other meta data
1. User Registration
2. Logging
User Service
JWT for session management
Cassandra DB
Text Data Storage for POST
Write Ops Fast
- blocked_post
-filtered_post
Post against policy
Content Service
- raw_post
-filtered_post
- blocked_post
TEMPORARY BUFFER
Images
Videos
Amazon S3
Returning a list of POSTS to Feed Service
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Generation of Feed
based on friends / followers
2. Feed service will check all the POSTS of other users from POST DB
3. Feed Service will return the response to the user
Feed Service
1. Retreving
Followers
Follower Service
Follower DB
User Engagment Service
Comment DB


Users
- userID
- Username
- Email
- Password
- Phonenumber
- Followers_count
- friends_count
- profile_url
- other meta data of user
User DB
PostgreSQL
Registration: Data Saving in DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Post (Document DB)
- post_id
- user_id
- post_type
- content_text
- media_url
- thumbnail_url
- like_count
- share_count
- comment_count
- other meta data
Text Data Storage for POST
Write Ops Fast
Post against policy
- blocked_post
-filtered_post
Content Service
- new_post
-filtered_post
- blocked_post
Images
Videos
Cassandra DB
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Returning a list of POSTS to Feed Service
2. Feed service will check all the POSTS of other users from POST DB
Generation of Feed based on friends / followers
3. Feed Service will return the response to the user
Feed Service
1. Retrieving Followers
Follower
- Follow_id (PK)
- Follower_id
- Following_id
- Status_of_req
- Timestamp
Follower Service
For now We only use Postgres
But if, the requirement is find FOF -> use Graph DB
User Engagement Service
Comment DB
Like DB


Engagement Service
Users
- userID
- Username
- Email
- Password
- Phonenumber
- Followers_count
- friends_count
- profile_url
- other meta data of user
User DB
PostgreSQL
Registration: Data Saving in DB
Check user's credentials
HTTP Response: User Verified
1. User Registration
2. Logging
User Service
JWT for session management
Post (Document DB)
- post_id
- user_id
- post_type
- content_text
- media_url
- thumbnail_url
- like_count
- share_count
- comment_count
- other meta data
Text Data Storage for POST
Write Ops Fast
Post against policy
- blocked_post
-filtered_post
Content Service
- new_post
-filtered_post
- blocked_post
Images
Videos
Cassandra DB
Clients
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Returning a list of POSTS to Feed Service
2. Feed service will check all the POSTS of other users from POST DB
Generation of Feed based on friends / followers
3. Feed Service will return the response to the user
Feed Service
1. Retrieving Followers
Follower
- Follow_id (PK)
- Follower_id
- Following_id
- Status_of_req
- Timestamp
Follower Service
KAFKA PRODUCER
We will not directly inject the like and comments as it becomes a heavy operation consuming SCM users
Pub-Sub Model
User Engagement Service Consumer
Comment
- comment_id
- post_id
- user_id
- like_count
- other data
Like
- like_id
post_id / comment_id
- user_id
- reaction_type
- other data


Feed Service
Users
User DB
PostgreSQL
Registration: Data Saving in DB
Check user's credentials
HTTP Response: User Verified
Post (Document DB)
1. User Registration
3. Logging
User Service
JWT for session management
Text Data Storage for POST
Write Ops Post
Post against policy
Content Service
Images
Videos
Cassandra DB
Check the post's from POST DB in prior & pass them to KAFKA queue.
API Gateway & Load Enhancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Clients
LARGE REDIS CACHE
For normal person
Read Post (POST)
2. Get the all recent post based on my followers
For a specific user storing all posts to show on feed.
Redis DB
2. write
Feed Service
Read
Feed Cache
Feed preparation in cache
Feed DB
Follower Service
Follower
Timestamp
Write heavy operations
Content
User Engagement Service
Consumer
Like
Pub Sub Model


Feed Service : further optimization
Users
User DB
PostgreSQL
Registration: Data Saving in DB
Check user's credentials
HTTP Response: User Verified
Post (Document DB)
1. User Registration
3. Logging
User Service
JWT for session management
POST DB
Text Data Storage for POST
Write Ops Post
Post against policy
Content Service
Images
Videos
Cassandra DB
Check the post's from POST DB in prior & pass them to KAFKA queue.
API Gateway & Load Enhancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Clients
For normal person
Read Post (POST)
2. Get the all recent post based on my followers
For a specific user storing all posts to show on feed.
Redis
2. write
Feed Service
Read
Feed Cache
Feed preparation in cache
Feed DB
Follower Cache (Top Followers)
2. Get the list of all followers in backfill
Read
Follower Service
Follower
Timestamp
Write heavy operations
Content
User Engagement Service
Consumer
Like
Pub Sub Model